# Parallelisation for HPC

Philip Blakely

Centre for Scientific Computing, University of Cambridge

# Parallelisation for HPC

# Outline


1. What is parallelism and why?
2. Parallelisation in Hardware
3. Hardware
4. How to exploit multi-core in software

# Parallel computing

- So far, all your programs have presumably been written serially.
- However, they probably do not run quickly enough to get you sufficient scientific results sufficiently quickly.
- (Where "sufficient" is defined by your supervisor's expectations and/or thesis/project deadlines.)

# Moore's Law

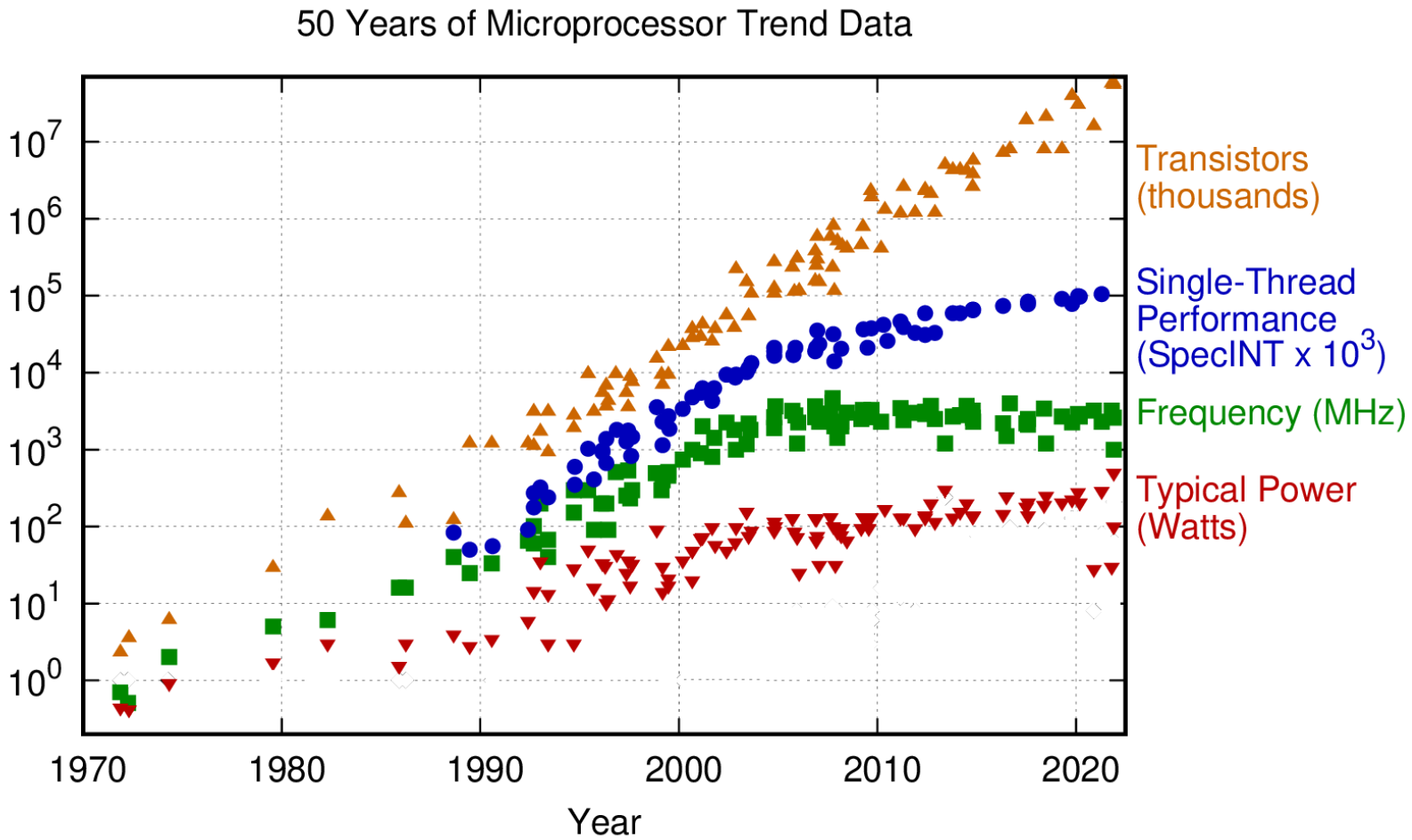
- You may have heard of:

**Moore's Law - 1965 (Gordon Moore: 1929-2023)**

The complexity for minimum component costs has increased at a rate of roughly a factor of two per year. Certainly over the short term this rate can be expected to continue, if not to increase. Over the longer term, the rate of increase is a bit more uncertain, although there is no reason to believe it will not remain nearly constant for at least 10 years.

- Note that this only suggests that the number of transistors on a processor will double every year. It does not suggest that processor performance will double every year.
- However, Moore's law has often been misquoted to say that processor performance will double every 18 months or so.

# Some historical data




Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

# Data analysis

- So, has Moore's law (or its misquoted version) held up?
- Between around 1970 and 2005, processor clock-speeds (governing the number of instructions that can be carried out per second) approximately doubled every 18 months.
- However, maximum processor speeds have stalled at around 3.5GHz since 2005, due to the problem of power dissipation at higher frequencies.
- Given that we want to run scientific programs faster and faster, what can we do?

# Parallelism

- The observant among you will have spotted that even though clock speeds (frequency) have not increased since 2005, the number of transistors has.
- So, what are these extra transistors doing?

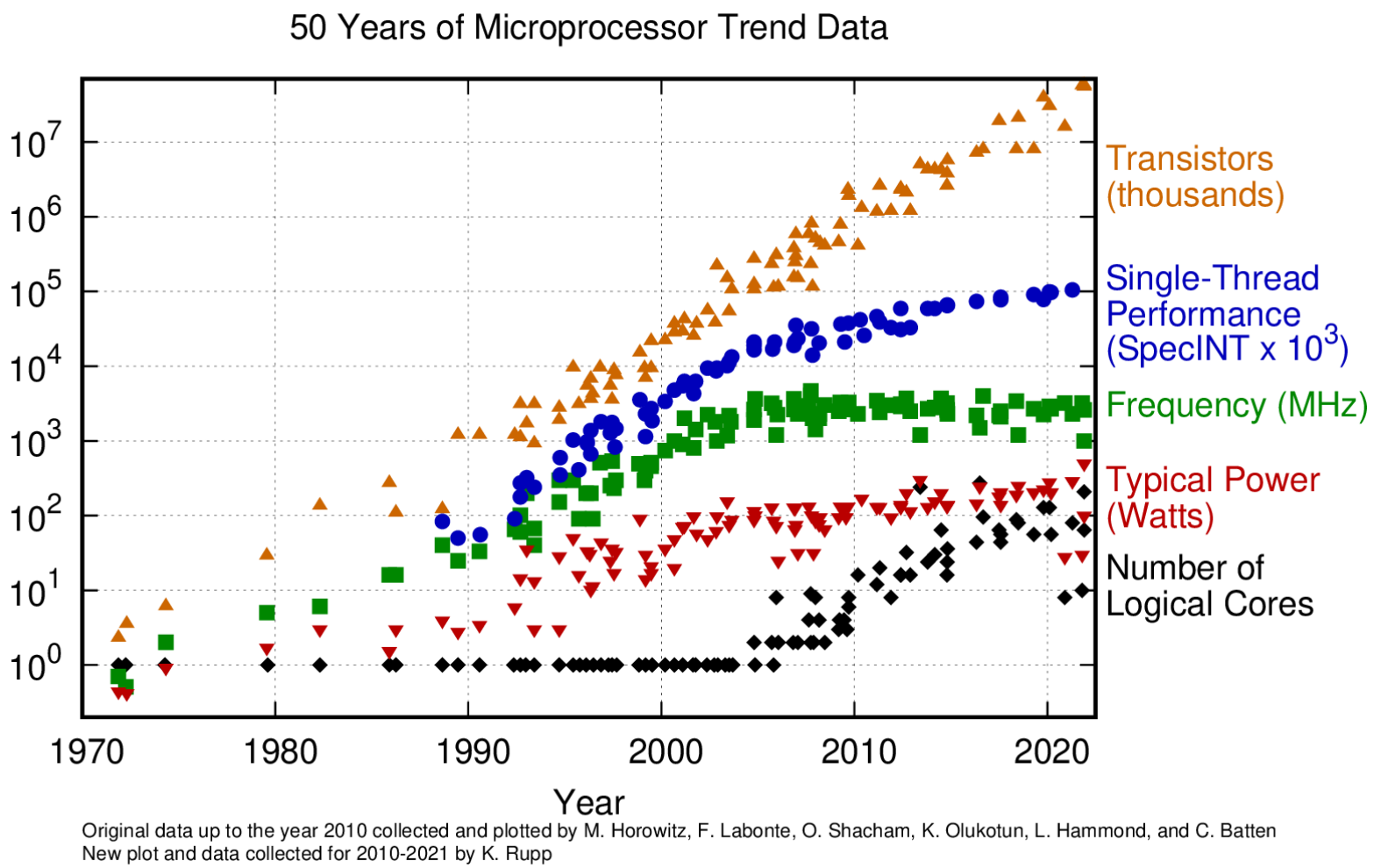
# Parallelism

- The observant among you will have spotted that even though clock speeds (frequency) have not increased since 2005, the number of transistors has.
- So, what are these extra transistors doing?
- Virtually all CPUs these days are "multi-core".
- This means that the CPU is capable of evaluating multiple separate streams of instructions simultaneously.
- This is part of the reason that your laptop/desktop can run multiple programs simultaneously without any loss in speed.

# Parallelism

- The observant among you will have spotted that even though clock speeds (frequency) have not increased since 2005, the number of transistors has.
- So, what are these extra transistors doing?
- Virtually all CPUs these days are "multi-core".
- This means that the CPU is capable of evaluating multiple separate streams of instructions simultaneously.
- This is part of the reason that your laptop/desktop can run multiple programs simultaneously without any loss in speed.
- (The other part of the reason is that most applications spend a lot of their time waiting for input from keyboard, mouse, or disk, and they can therefore allow other applications to run without any *apparent* loss of responsiveness. This would happen even on a single core processor, via "time-slicing".)

# Multi-core processors




Plot from `https://github.com/karlrupp/microprocessor-trend-data`
CCBY 4.0 `https://creativecommons.org/licenses/by/4.0/`
(Earlier plot had logical-core trend removed.)

# O/S Support

- These multi-core processors are supported by all modern O/S (Windows/Linux/Mac OS X/Android).
- Processes are scheduled onto whichever core(s) the O/S decides are most appropriate.
- At the hardware level, cores may share some features.
- E.g. data and instruction caches (caching main-memory) may be shared between cores in a hierarchical fashion.
- Some manufacturers (Intel) have hyper-threading, where two logical processors share one core, but have separate process-state storage.
- If one thread is stalled waiting for data from memory, the other thread can continue on the same core.
- As the threads share a single FPU, this is unlikely to yield helpful performance for scientific computing applications.

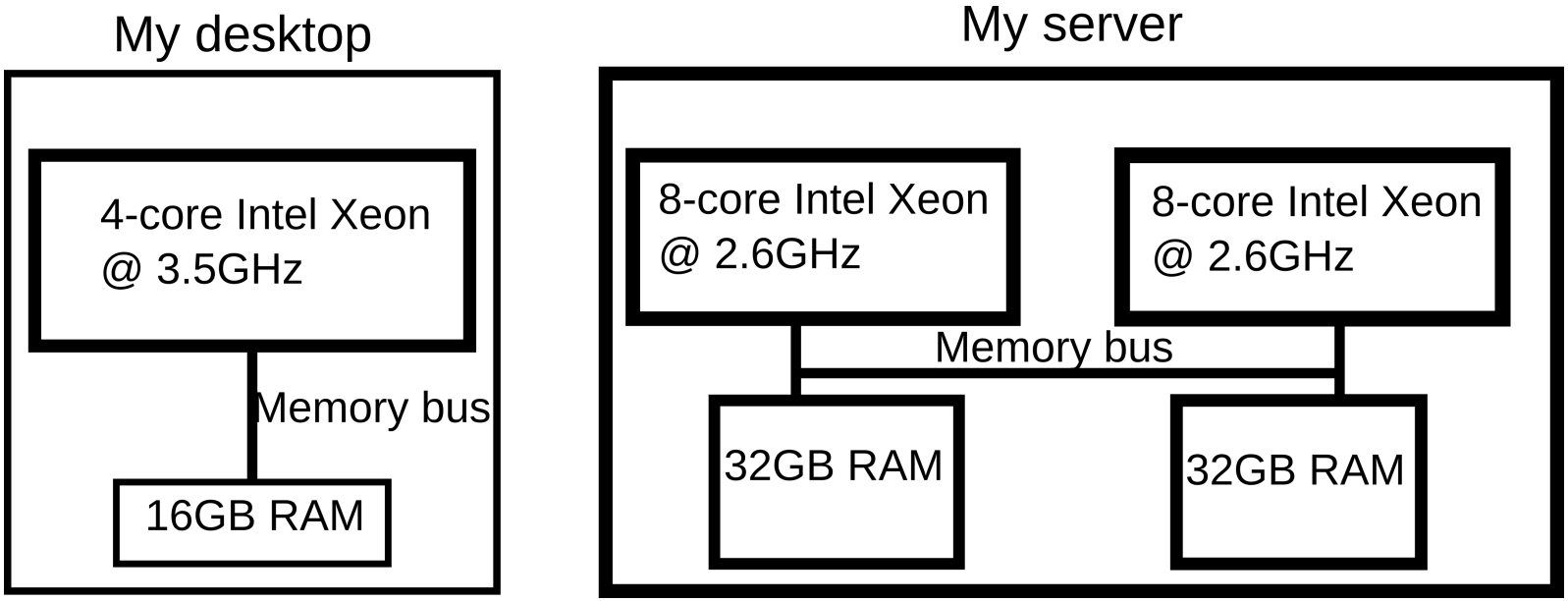
# Outline


1. What is parallelism and why?
2. Parallelisation in Hardware
3. Hardware
4. How to exploit multi-core in software

# Shared-Memory Parallelism

- For a single multi-core computer, all its internal memory is accessible directly to all cores on that processor.
- Linux/Windows/Mac OS X easily allow separate threads to share data.
- So we can run multiple copies of the same program on different cores, all having access to the same block of memory.
- The data should then be processed more quickly.
- There are also machines with more than one multi-core processor, but still sharing the same memory and OS.
- So-called dual-socket machines are common; quad-socket is somewhat rarer, and larger shared-memory machines tend to be custom-built (e.g. by Cray).

# Shared-memory architecture



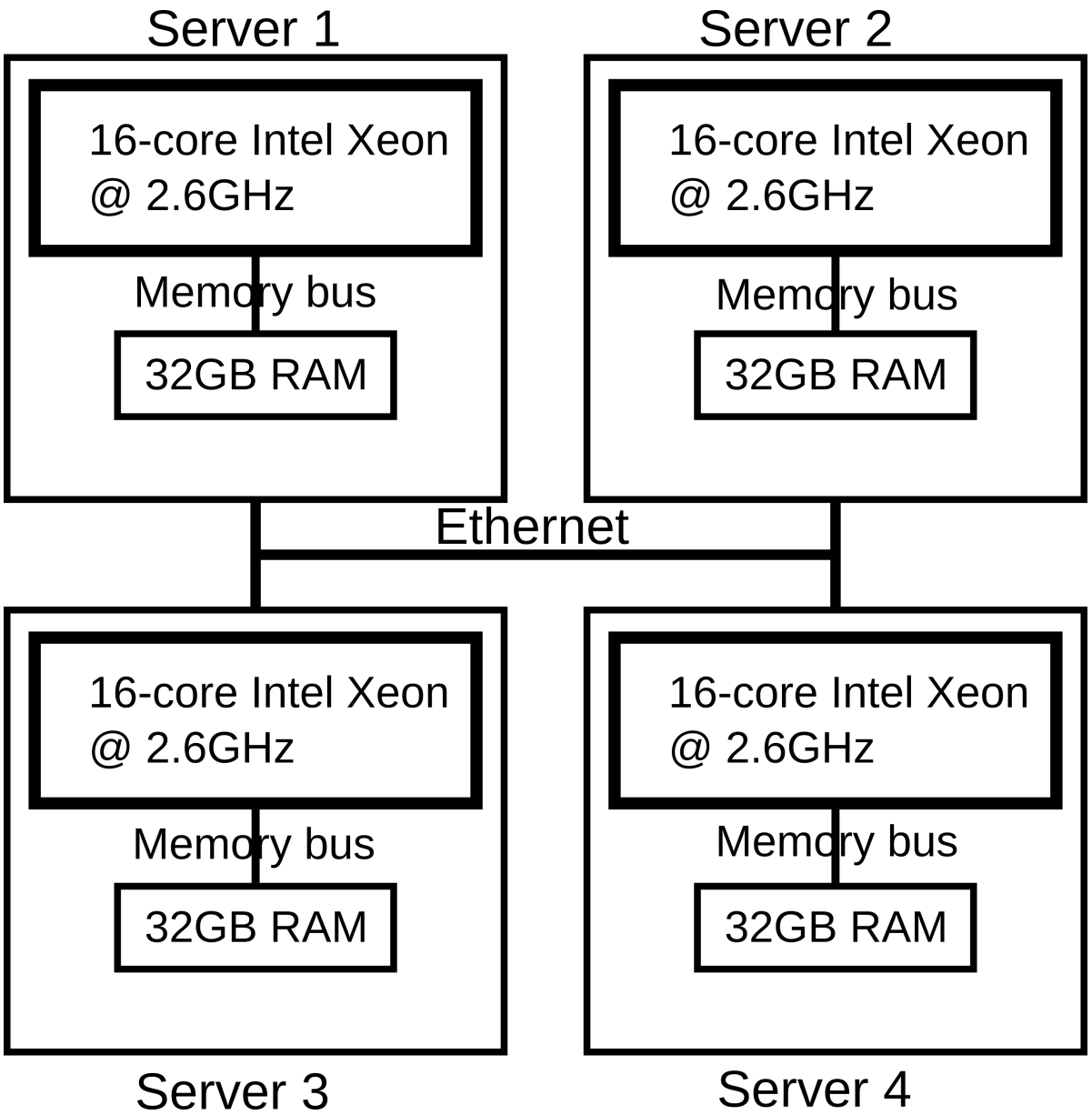
# How many cores?

- Currently, the largest number of cores Intel sell on one processor is 144; for AMD it is 192.
- The maximum amount of RAM in one server is around 3TB.
- These machines are very expensive (around £20k), and you may only have access to machines with a mere 32 or 76 cores and 128GB RAM.
- We will probably need our code to scale across more cores than available in one server; how can we do this from a hardware point of view?
- If we want to run programs across 100s of cores, and with access to over 3TB of memory, can we try another approach?

# Distributed Memory Parallelism

- What if we could connect multiple (fairly powerful) servers together with some form of *inter*connecting *net*work?
- Each server could run its own operating system, and only have access to its own block of memory, but could communicate with the other servers, so that the total amount of memory available was much larger.
- We would need some way of passing messages across the network.
- This approach still costs a lot of money, but can typically use off-the-shelf hardware and get fairly good performance.

# Distributed-memory architecture



# Caveats

- The preceding diagrams are very much simplified.
- Much more hardware (CPU-caches, PCI buses, network devices) is required in both cases.
- The distributed approach looks more complex, but is somewhat easier to scale up to a large number of processors, by just adding more off-the-shelf servers.
- It is often cheaper to buy multiple lower-performance servers than one big one.
- Reasonable performance can come from standard ethernet; much better performance comes from more advanced interconnect such as Infiniband.

# Outline


1. What is parallelism and why?
2. Parallelisation in Hardware
3. Hardware
4. How to exploit multi-core in software

# Interrogating your computer

```
$ lscpu
Architecture:            x86_64
CPU(s):                  24
On-line CPU(s) list:     0-23
Vendor ID:               GenuineIntel
Model name:              Intel(R) Xeon(R) W-2265 CPU @ 3.50GHz
Thread(s) per core:      2
Core(s) per socket:      12
Socket(s):               1
CPU max MHz:             4800.0000
CPU min MHz:             1200.0000
L1d cache:               384 KiB (12 instances)
L1i cache:               384 KiB (12 instances)
L2 cache:                12 MiB (12 instances)
L3 cache:                19.3 MiB (1 instance)
NUMA node(s):            2
NUMA node0 CPU(s):       0-2,6-8,12-14,18-20
NUMA node1 CPU(s):       3-5,9-11,15-17,21-23
```

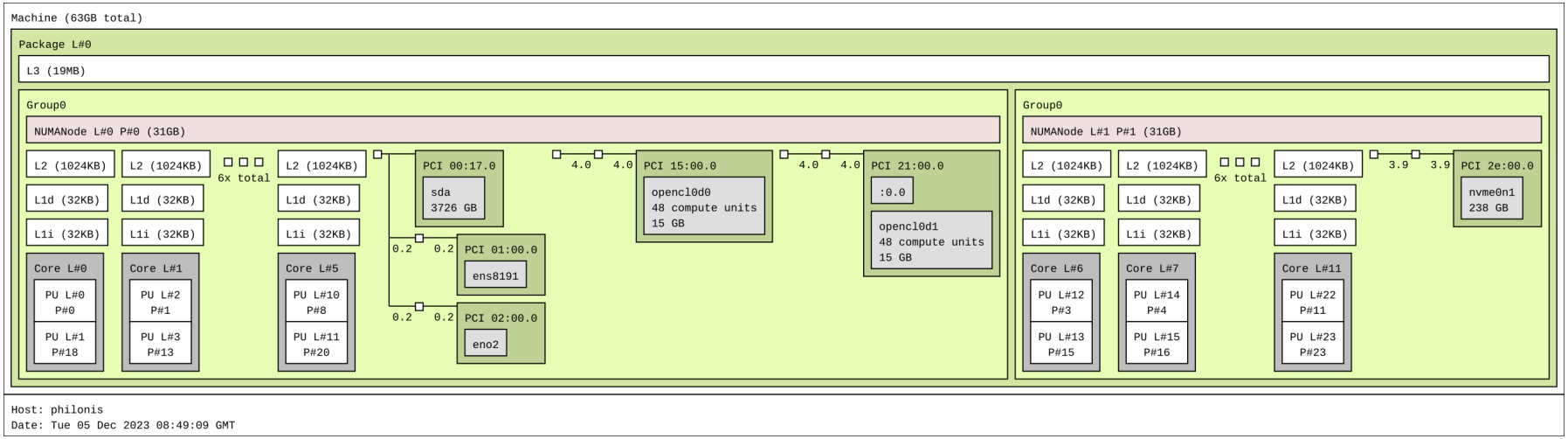
# Interrogating your computer

Available memory:

```
$ free -m
        total    used    free  shared buff/cache available
Mem:    64116   18731    4941     340     41860     45384
Swap:   16383    3557   12826
```

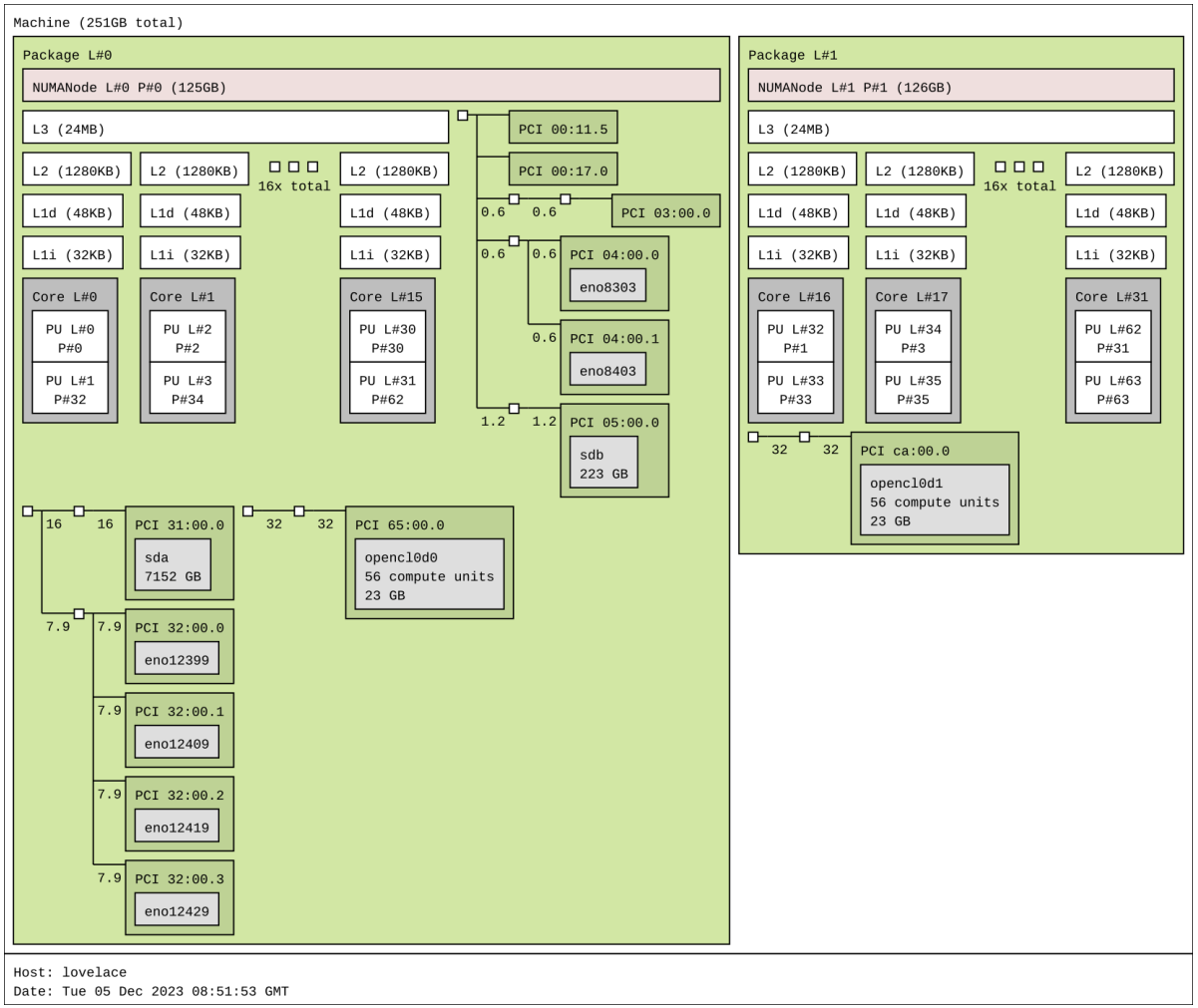
Hardware topology:

```
philonis:~$ lstopo --of svg > philonis_lstopo.svg
```



# lstopo

```
lovelace:~$ lstopo --of svg > lovelace_lstopo.svg
```



# Outline


1. What is parallelism and why?
2. Parallelisation in Hardware
3. Hardware
4. How to exploit multi-core in software

# Software solutions

- We now know (sort of) how to build and obtain hardware with 100s to 1000s of CPU cores.
- But we still need some way of telling the computer which bits of our code run on which processors.
- For the Distributed-memory architecture approach, the *de facto* approach is MPI (Message Passing Interface).
- For the Shared memory architecture, MPI will still work, but you can also use OpenMP®.
- Alternatives such as Co-Array Fortran and UPC++ exist, but are less common.

# Distributed vs shared-memory parallelism programming

- Distributed parallelism:
  - Uses separate processes
  - Usually requires explicit work on the programmer's part.
  - Work done is explicitly divided between different processes
  - Data must be explicitly passed between processes.
  - This usually requires rewriting a serial code from scratch.
  - Extra work usually pays off as the code can be divided onto different compute-servers.
- Shared-memory parallelism:
  - Compiler/library divides work automatically between threads.
  - Still requires some hints from the programmer, but is more straightforward.
  - Data does not need to be passed explicitly (although of course it is at a fundamental hardware level).
  - Cannot go beyond a single compute-server.
- The above are slight generalisations and depend on the particular parallel framework being used.

# MPI (Message Passing Interface) - Distributed

- MPI is an API (Application Programmer Interface) that allows different processes to pass data between them.
- It is (usually) implemented via special function calls to an external library.
- We run multiple copies of the same executable simultaneously (which then pass data between themselves).
- Each executable will run on a different CPU core (subject to OS scheduling).
- Each separate process will solve one part of the problem and communicate its results as necessary to the others.
- MPI requires explicit function calls to pass specific blocks of data.
- Each separate copy of the executable only has direct access to its own data. The only way it can access another process's data is by being passed it explicitly.

# OpenMP® - Shared

- OpenMP® is an API defined via 'pragmas'; special instructions embedded in an application's source code, understood by the compiler.
- With OpenMP, we launch a single copy of the program only, which then 'forks' into multiple threads which then communicate via shared memory (as well as having their own private memory region).
- Separate threads run on separate CPU cores (subject to OS scheduling).
- Typically, separate threads will run algorithmically independent iterations of loops.
- All threads have access to the same block of data.
- Care has to be taken to ensure that multiple threads don't try to modify the same data in memory simultaneously.

# OpenMP versus MPI

- Advantages of OpenMP:
  - If you have a serial C/C++/Fortran code, you can add a few simple OpenMP "pragmas" and speed your code up fairly well on a multi-core laptop (as well as single nodes of a compute cluster).
  - The compiler/OpenMP library schedules computations and synchronises threads while the hardware handles all data communication between threads.
  - You do not have to worry (too much) about which threads carry out which calculations.
- Disadvantages of OpenMP:
  - If you are not careful, you have "race-conditions" where multiple threads try to update the same memory location simultaneously. The results depends on which thread runs first; this is not under the control of either you or the O/S.
  - You can only scale across one server, i.e. a maximum of 76 cores on CSD3 (hardware-imposed single-node limit).

# MPI versus OpenMP

- Advantages of MPI:
  - MPI parallelism works across multiple nodes of a supercomputer, e.g. up to 4,256 cores on CSD3 (sysadmin-imposed limit).
  - You have very explicit control over which processes act on which bits of data, and how data is distributed.
  - MPI libraries exist for C, C++, Fortran, Python, and Java.
- Disadvantages of MPI:
  - You more-or-less need to rewrite a serial algorithm from scratch to take advantage of MPI.
  - You can still have race-conditions and the possibility of lock-up where two (or more) processes are waiting for each other.

# GPUs

- GPUs are somewhat different again, and have their own programming approach.
- They are "co-processors" and are physically separate from the CPUs.
- They require slightly more advanced understanding than MPI/OpenMP but many of the same programming principles hold.
- The CUDA language only works for C++ and Fortran, requires an good understanding of GPU hardware and is specific to NVIDIA GPUs.
- Other GPU-directed languages exist (including OpenMP).

# Parallel Programming and Performance

# Outline


5 Practical issues

6 Theoretical performance

7 Parallelisation errors

8 Where to find parallel hardware

9 Alternatives

10 Clusters

11 CSD3 specifics

# Embarassingly parallel problems

- Some scientific problems are very easy to parallelise.
- For example, performing a parametric study for a problem requiring the solution of a single ODE.
- If there are thousands of parameter combinations, then just run thousands of copies of a serial ODE solver.
- They are completely independent, so can be solved simultaneously without any parallel programming at all.
- The O/S can schedule all the simulations and distribute them across multiple cores.
- This is an embarassingly parallel problem, solvable with a Bash/Python script (probably).
- (Don't just launch 1000 processes at once; consider using Bash's `parallel`).

# Latency and bandwidth

- Parallelism is not a magic bullet. Transferring data from one point to another takes time, whichever parallelism framework you use.
- The hardware required to transmit the data (network switch or shared-memory interconnect) adds more overhead.
- A typical minimum time to transmit the smallest message might be $3\mu$s (latency).
- The maximum speed of data transmission might be 10 giga-bit per second (or roughly 1GB/s) (bandwidth).
- Custom hardware will improve on these (e.g. Cray's memory interconnect or Infiniband™ instead of ethernet), but physical limits will always apply.
- As a rule-of-thumb, remember that light takes 1ns to travel 30cm (roughly the size of a desktop/server), and a 3.3GHz processor can carry out 3 operations in that time.

# Other hardware factors

Other hardware factors that will affect scaling are:

- Shared CPU caches: If instruction/data-caches are shared between CPU-cores, then these can effectively be used for low-latency message-passing although this is outside the programmer's direct control.
  (Low-level pinning of processes/threads to cores is possible, but is more advanced.)
- CPU performance: If you have a particularly fast processor, it is better to write code that spends more time on calculation than on data-passing. Vice-versa for a slow processor.

However, these are more incidental; the primary factors are latency and bandwidth.

# Outline


5 Practical issues

6 Theoretical performance

7 Parallelisation errors

8 Where to find parallel hardware

9 Alternatives

10 Clusters

11 CSD3 specifics

# Algorithms

- If programming for parallel architectures, consider your algorithm and hardware carefully.
- The optimal algorithm for a serial code may not be easy to parallelise efficiently, and a simpler (but slightly slower) one may be easier to parallelise.
- Depending on the parallel architecture, different algorithms may be preferable.
- For example, consider the extremes of very slow and very fast data communication.
- Read the available literature for your field, including possibly 1970s papers.
- Some thinking may be required to extract the fundamentals of your algorithm in case someone has solved a similar problem but not published it in exactly the same terminology you use.

# Amdahl's Law

- As a very simple approximation, the time to complete a parallel program is:

$$t_n = t_s + \frac{t_p}{N}$$

  where $t_s$ is the time taken for the serial part of the code, $t_p$ is the time taken for the parallelisable part of the code, and $N$ is the number of processors available.
- Even if 95% of your code (by run-time) can be parallelised, the maximum speed-up possible is a factor of 20.
- Amdahl's Law can also be written as:

$$\begin{aligned} \text{Speed-up} &= \frac{\text{Serial run-time}}{\text{Parallel run-time}} \\ &= \frac{s+p}{s+p/N} \\ &= \frac{1}{s+p/N} \end{aligned}$$

  where $s + p = 1$ breaks the run-time down into serial and parallel parts.

# Amdahl's law extended

- If we include the cost of message passing, then:

$$t_n = t_s + \frac{t_p}{N} + c\lambda N$$

  where $\lambda$ is the time taken to pass data between two processes, and $c$ is a constant giving the number of messages per processor.
- We have assumed that the number of messages scales with $N$, which is likely for most non-trivially-parallelisable problems.
- This is no longer monotonically decreasing as $N \to \infty$.
- Remember that even though OpenMP does not have explicit message passing (unlike MPI), data still has to be passed between cores.

# Gustafson-Barsis's Law

- Amdahl's Law assumes that the proportion of the run-time that is parallelisable is fixed.
- However, if we allow our problem size to grow with the number of cores, then the parallelisable part is $pN$ so that:

$$\begin{aligned}\text{Speed-up} &= \frac{\text{Serial run-time}}{\text{Parallel run-time}} \\ &= \frac{s+pN}{s+p} \\ &= s + pN \\ &= N + s(1-N)\end{aligned}$$

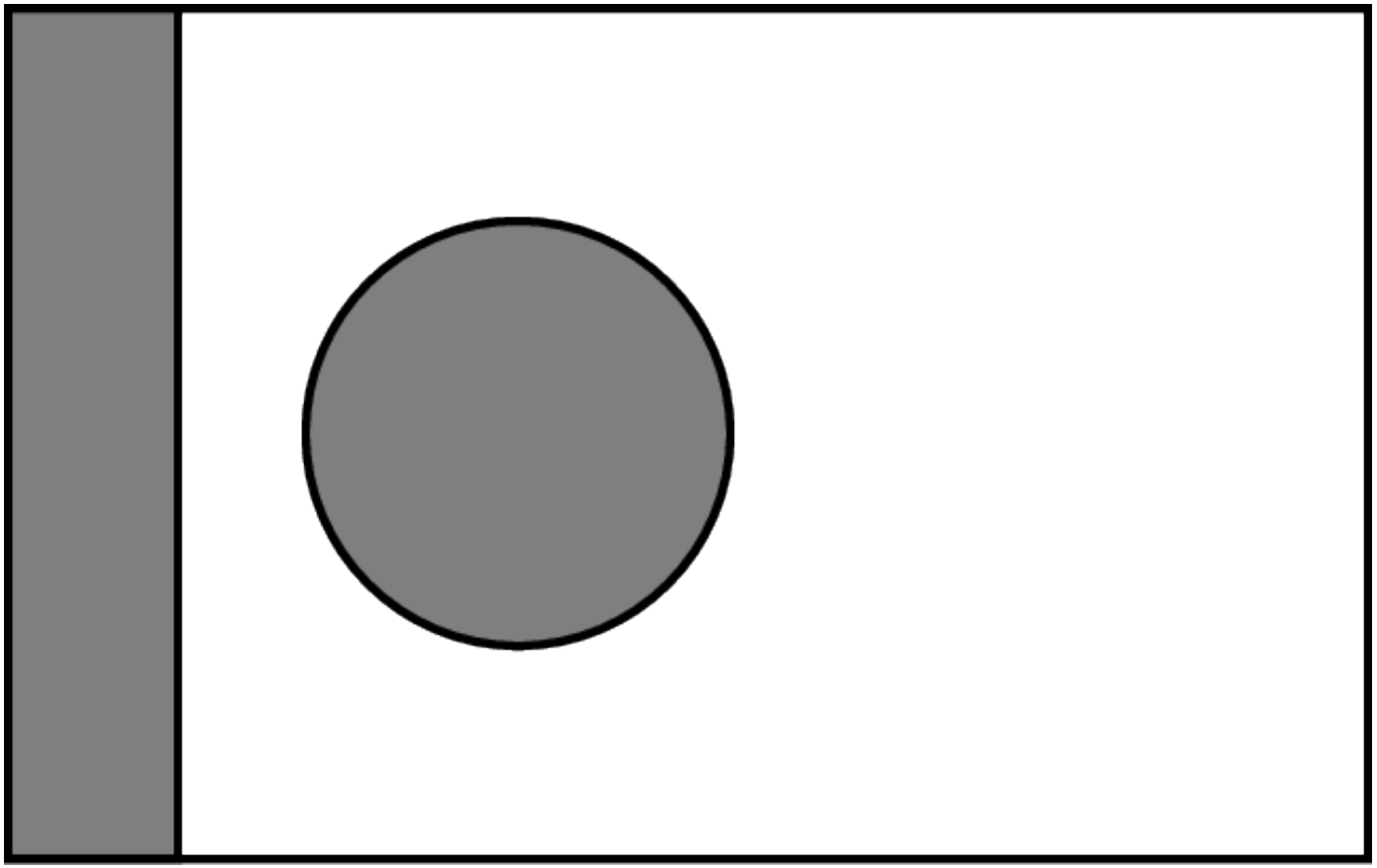
  See Gustafson "Reevaluating Amdahl's law": `https://doi.org/10.1145/42411.42415`

- The point is that the "amount of work that can be done in parallel varies linearly with the number of processors".

# Strong and weak scaling

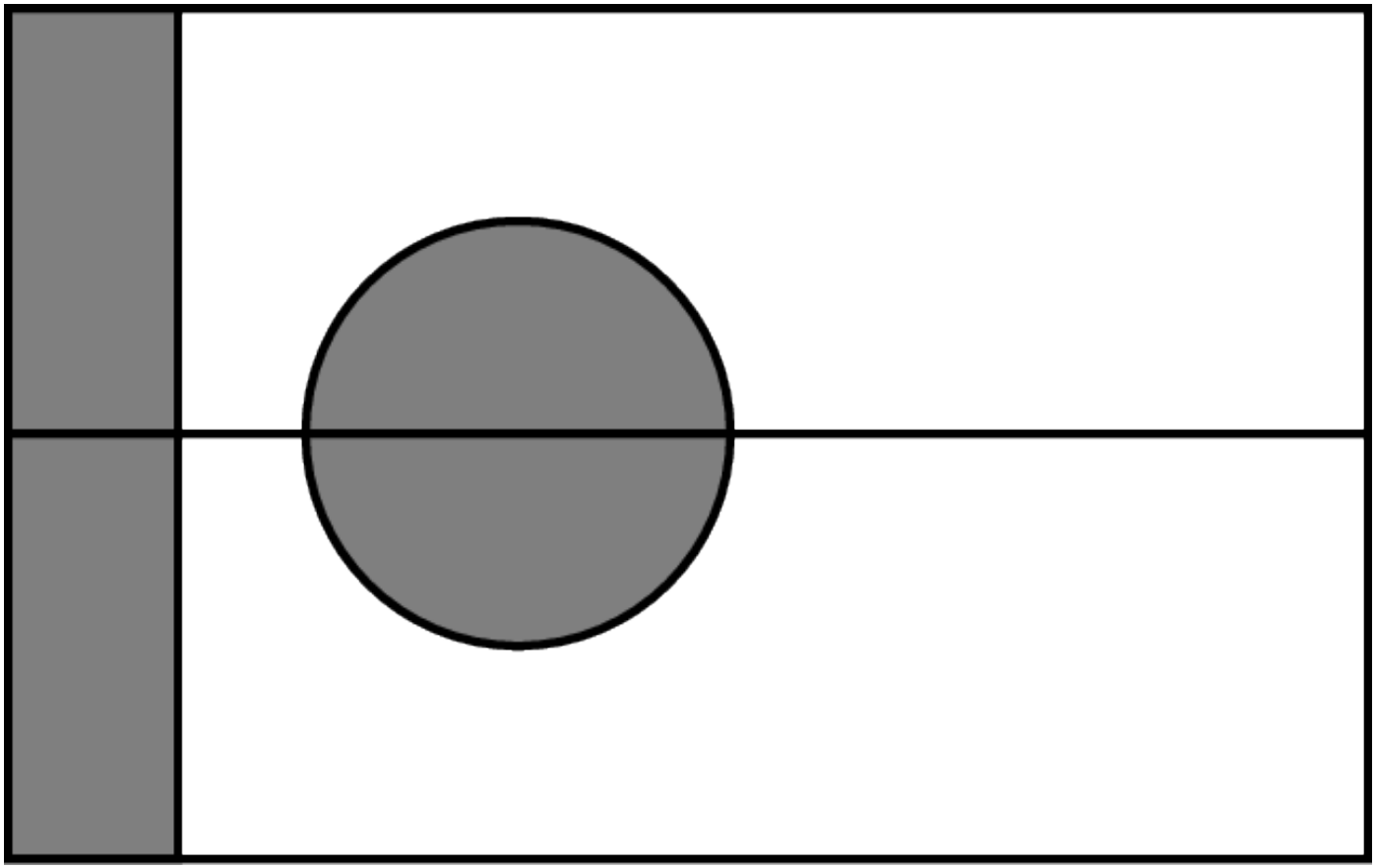
- When evaluating parallel performance, there are two ways to test it.
- "Strong scaling" requires you to measure the time taken to solve a problem of size $N$ on $n$ processors for $n = 1, 2, 3, \ldots, N$.
- "Weak scaling" requires you to measure the time taken to solve a problem of size $n$ on $n$ processors, for $n = 1, 2, 3, \ldots, N$.
- Strong scaling is harder to get good performance for, but should be aimed for if at all possible (e.g. on a quad-core machine you can solve a problem in a quarter of the time).
- Weak scaling can be easier to attain and may be more realistic for research (e.g. on a quad-core machine you can now solve a problem four times as large in the same time as the original problem).

# Strong scaling - illustration



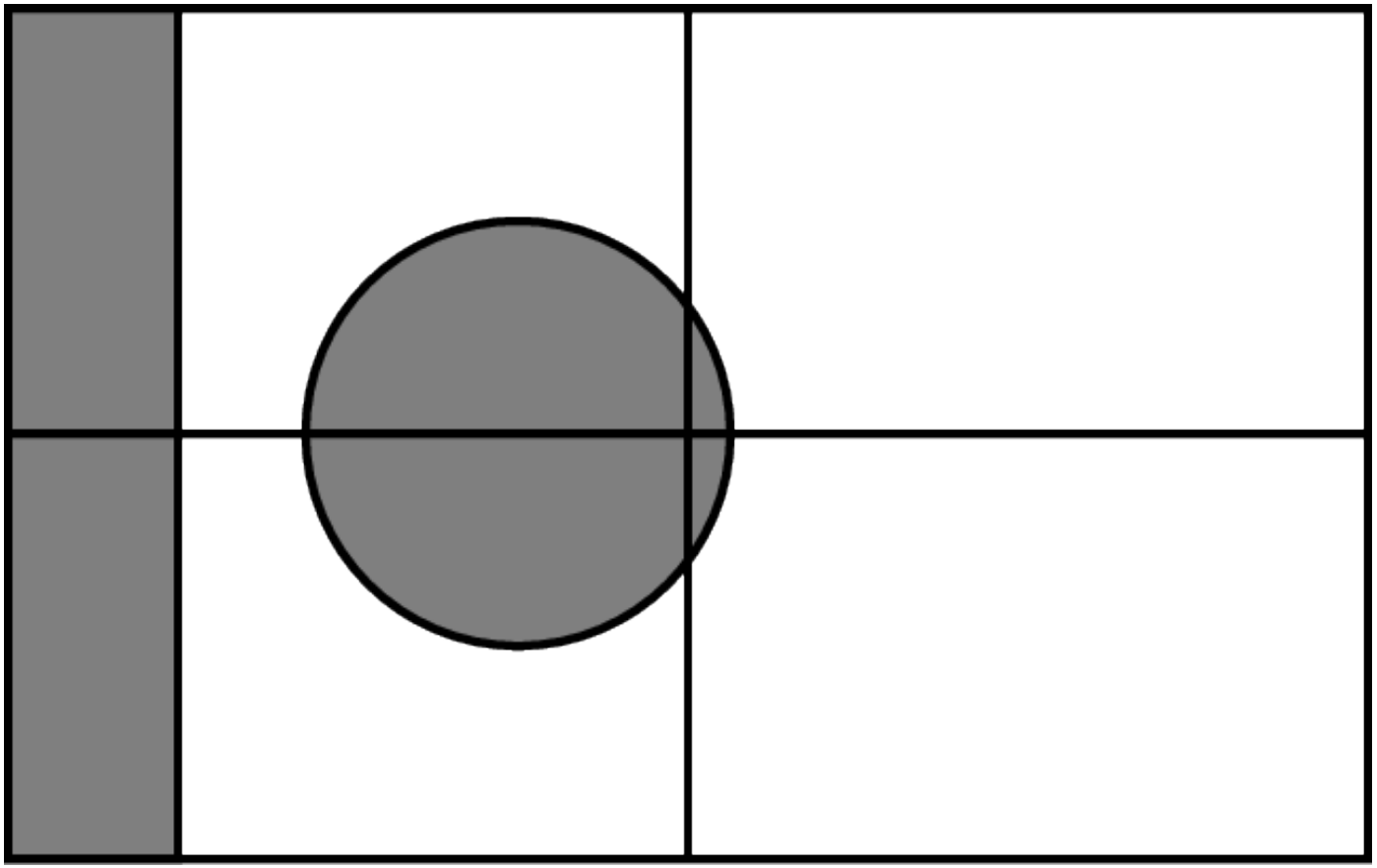
The amount of work per processor decreases, increasing the proportion of communication as a fraction of the whole simulation.

# Strong scaling - illustration



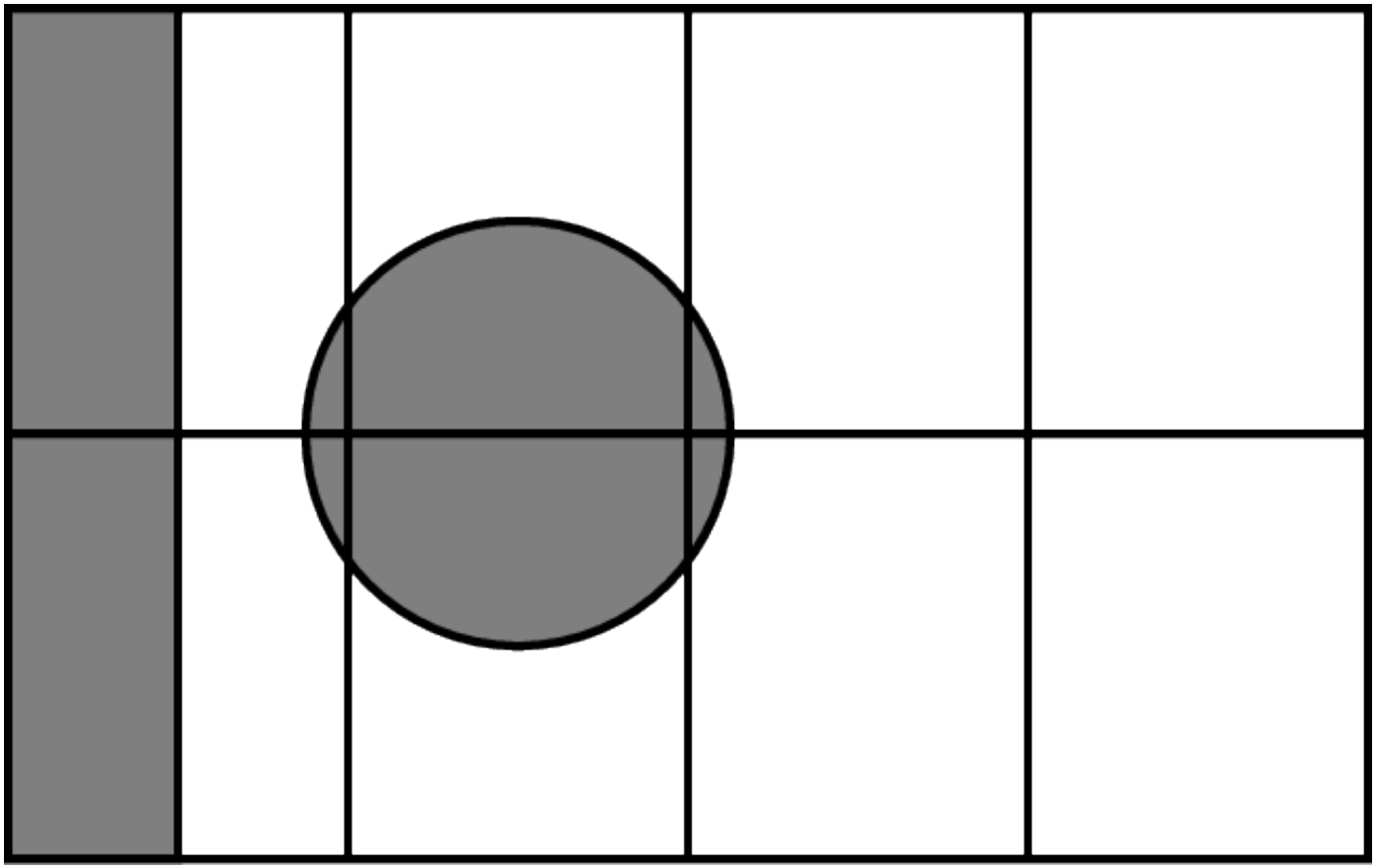
The amount of work per processor decreases, increasing the proportion of communication as a fraction of the whole simulation.

# Strong scaling - illustration



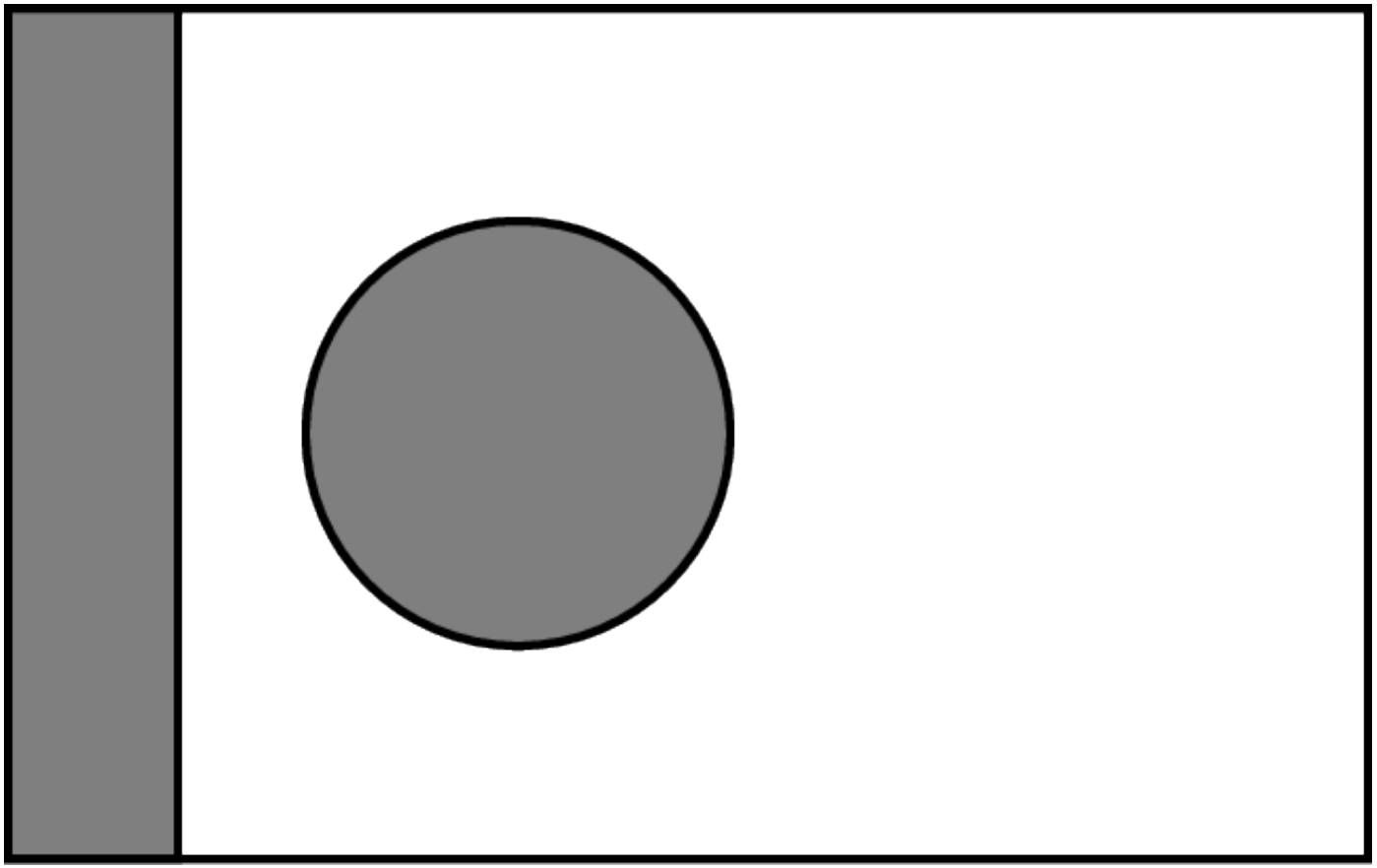
The amount of work per processor decreases, increasing the proportion of communication as a fraction of the whole simulation.

# Strong scaling - illustration



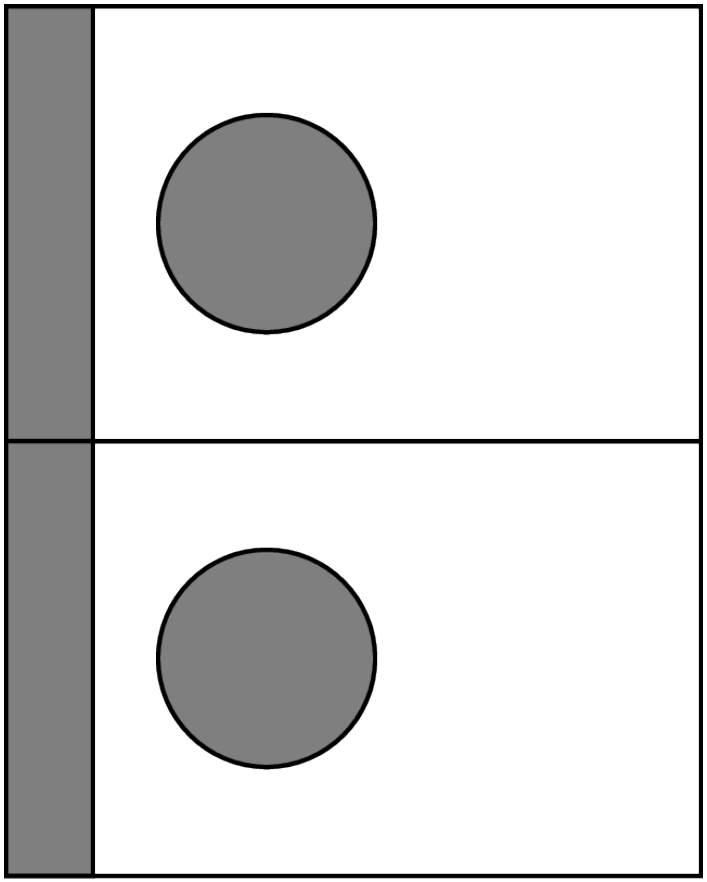
The amount of work per processor decreases, increasing the proportion of communication as a fraction of the whole simulation.

# Weak scaling - illustration



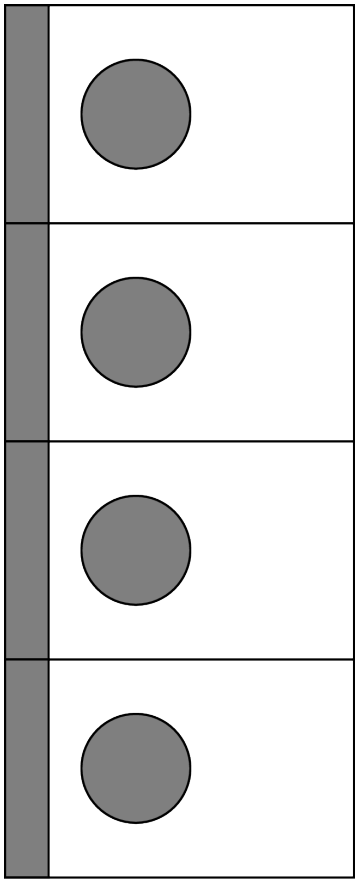
The amount of work per processor is constant, allowing the effect of communication alone to be seen.

# Weak scaling - illustration



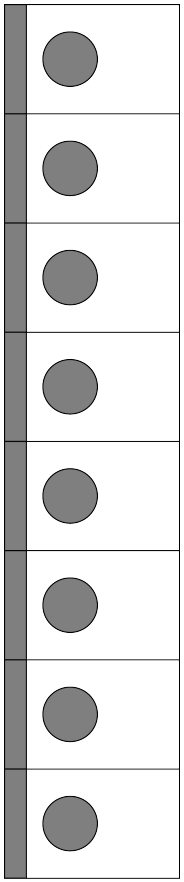
The amount of work per processor is constant, allowing the effect of communication alone to be seen.

# Weak scaling - illustration



The amount of work per processor is constant, allowing the effect of communication alone to be seen.

# Weak scaling - illustration



The amount of work per processor is constant, allowing the effect of communication alone to be seen.

# Outline


5 Practical issues

6 Theoretical performance

7 Parallelisation errors

8 Where to find parallel hardware

9 Alternatives

10 Clusters

11 CSD3 specifics

# Errors in parallelism

- There are two main types of error in parallelism:
  - The logic of your algorithm means that multiple processes end up waiting for each other - usually obvious as the program locks up.
  - A race condition occurs when the data you read from memory depends on some other condition (e.g. whether data has been communicated) and you have not enforced that dependency. This can be tricky to detect as often the results look plausible...
  - With race conditions, the results may change each time you run it (but may not) as the order that threads access the data may vary.
  - These thread-timings can depend on what other processes or data are making use of the CPU at the same time.
- Other forms of these errors exist, and can be very hard to debug.
- Sometimes errors only emerge when using particular numbers of cores, or multiple nodes.
- You should understand where your data is at all times, and also understand what state your program could get into (even if it seems unlikely).

# Deadlocks

- A deadlock occurs when one process or thread is waiting for a message or signal from another before it can progress, but that will never happen due to a logic bug.
- Or if two or more threads are each waiting for each other to send/receive a message (possibly in a cyclic fashion).
- Therefore, no processes can logically progress and the program deadlocks.
- Deadlocks tend to be the result of explicit logic bugs on the part of the programmer.
- At least they usually consistently lock-up as the logic is under the programmer's control, even if this only happens for (say) an odd number of processes.

# Debugging in parallel

- Debugging is hard at the best of times.
- Debugging in parallel is even harder.
- Commercial parallel debuggers such as TotalView and DDT exist.
- A free/open-source parallel debugger is `mdb`
  `https://github.com/TomMelt/mdb`
- Try to reproduce the problem on as few processors as possible, for the simplest possible case.
- One approach is to output data from all processes with print statements, prefixed by the processor number (for example).
- I have often resorted to running multiple *serial* debuggers simultaneously and stepping through them, or examining program state on a deadlock.

# Making best use of hardware

The following applies to all computing systems in general but more-so to clusters where your funds are more likely limited.

- Ensure that your algorithm is (more-or-less) the most optimal for your problem and hardware (finding the perfect algorithm is either impossible or a research problem itself).
- Write your code so that it load-balances reasonably across all hardware available to it (i.e. all cores are doing roughly the same amount of work).
- Run your tests on a suitable number of cores, given the problem-size and the measured scaling behaviour of your code.

# Outline


5 Practical issues

6 Theoretical performance

7 Parallelisation errors

8 Where to find parallel hardware

9 Alternatives

10 Clusters

11 CSD3 specifics

# On your desk

- Every laptop these days has multiple CPU cores.
- CSC laptops have 4 cores; laptops up to 20 cores are available.
- Apple Macbook Pros have 10-12 cores.
- A typical desktop PC will have 12-24 cores, perhaps more.
- CSC has a range of desktops and servers with 4-32 cores (48-cores coming soon...)

# Clusters and Super-computers

- Individual research groups, departments, universities, and countries have their own supercomputers.
- Private companies have their own internal systems, as do government organisations.
- The most powerful supercomputers are listed at `https://www.top500.org/`.
- The list includes academic super-computers, and any commercial systems whose owners submit performance statistics. It does not include systems whose owners wish to keep their system a (state/industrial) secret.
- The list is currently headed by the U.S.A., with Germany, Italy, Japan, and Switzerland following shortly behind (U.K.'s first entry Isambard (Bristol) is at 11, then Njoerd at 26; Cambridge's DAWN is at 82).
- For your assignments and projects you may be given access to CSD3 depending on projected need.

# Outline


5 Practical issues

6 Theoretical performance

7 Parallelisation errors

8 Where to find parallel hardware

9 Alternatives

10 Clusters

11 CSD3 specifics

# Vectorization - sort-of parallelism

- Most modern processors have the ability to carry out a single instruction on multiple pieces of data simultaneously on one core.
- For example, computing the sum of two four-vectors: $\mathbf{c} = \mathbf{a} + \mathbf{b}$.
- This is known as vectorization. More complex operations are available, but are baked into the CPU.
- This is parallelism to some degree, but requires your data and algorithm to be in a very particular form.
- This vectorization is usually done by the compiler, based on its knowledge of the processor and your data layout.
- This only covers very specialised cases, and for a very small degree (e.g. 8 doubles) of parallelism.
- For more details, see Advanced Research Computing.

# Outline


5 Practical issues

6 Theoretical performance

7 Parallelisation errors

8 Where to find parallel hardware

9 Alternatives

10 Clusters

11 CSD3 specifics

# Using a cluster

- So far you have only used single-node systems; how can you run on multi-server systems?
- An example is Cambridge's CSD3 (Cambridge Service for Data Driven Discovery)
- Full documentation is at `https://docs.hpc.cam.ac.uk/hpc/`
- Run-time is cheap(ish) at 1p per core-hour (AWS is around 8p per core-hour).
- GPU-enabled nodes are available for a higher cost.
- The main problem is that everyone wants to use CSD3 simultaneously.
- For this, there is a complex queueing and accounting system.

# Clusters in general

In order to support a wide range of researchers, clusters will have been installed with some or all of the following:

- Multiple compilers, to support multiple languages, and as different compilers may give different code performance.
- Numerical libraries such as BLAS/LAPACK/etc. that are better optimized for the architecture than your own algorithm.
- High-bandwidth/low-latency interconnect between compute-nodes to improve data-passing performance.
- GPU/multi-core processors to accelerate calculations.

# Clusters in general ctd.

- A queueing system that ensures full and equitable sharing of resources between researchers.
- Shared file-systems between login and compute nodes so that the same environment exists on all nodes. These will have quotas to stop accidental filling of disks.
- The file-systems will also be fairly robust to hardware failure and distributed across 100s of disks and able to handle high-bandwidth parallel I/O.
- Separate login and compute nodes to allow for interactive compilation and debugging without waiting for the queuing system.
- Remote visualisation nodes to allow post-processing without having to transfer data back to local machines.

# Queueing notes

- Always give a good estimate of the total run-time of your simulation.
- If you don't ask for enough time, it will be killed early.
- If you ask for too much, your simulation may not run for a while.
- Understanding the queueing system can allow you to make best use of the cluster.
- There are limits on maximum job-length (e.g. 12-36 hours) and maximum number of nodes (e.g. 70 nodes).
- You may need to implement checkpointing in your code to run longer simulations.

# Debugging on a cluster

- Usually, you should debug on a local laptop/server; interactive use is the default.
- However, if the test-case is large enough (and you can't simplify the problem) then debugging on a cluster may need an interactive session.
- Short (1-2 hour) jobs can be run giving you direct/interactive access to a compute node.
- Undesirable as if you have a lunch-break half-way through, the clock is still ticking...
- `https://docs.hpc.cam.ac.uk/hpc/user-guide/interactive.html`

# Data post-processing

- Once your simulation is finished, you have to analyze the output.
- It is usually best to analyze it in-place as transferring 10s of GB of data to your local laptop will take a long time.
- Perhaps you need to run/create a parallelised post-processing tool.
- You can launch a remote desktop (*not* `ssh -X` !) on CSD3 and run your visualisation software there.
- See CSD3's documentation on visualisation for more details: `https://docs.hpc.cam.ac.uk/hpc/user-guide/turbovnc`

# Outline


5 Practical issues

6 Theoretical performance

7 Parallelisation errors

8 Where to find parallel hardware

9 Alternatives

10 Clusters

11 CSD3 specifics

# CSD3 specifics

- CSD3 has 8 login nodes accessible at `login.hpc.cam.ac.uk`
- CSD3 also has 544 compute nodes not directly accessible to users.
- You compile your code on the login nodes, and then submit long-running jobs to a queueing system to run on the compute nodes.
- **Never** run simulations on login nodes; the sys-admin will send nasty emails (or just kill your jobs outright).
- Very short (less than 1 minute) test-runs are probably OK, just to check everything is compiled properly.

# Modules

- On clusters, there are many compilers (companies and versions), MPI implementations, Python versions, etc. How can we keep track?
- On a simple Linux system you would modify your `PATH` and `LD_LIBRARY_PATH` variables.
- You can do this on CSD3 but you will lose track of everything.
- On CSD3 you can use `module` command to keep track.
- `$ module list`

```
Currently Loaded Modulefiles:
  1) dot                                  9) intel/impi/2020.2/intel
  2) slurm                               10) intel/libs/idb/2020.2
  3) turbovnc/2.0.1                      11) intel/libs/tbb/2020.2
  4) vgl/3.1/64                          12) intel/libs/ipp/2020.2
  5) singularity/current                 13) intel/libs/daal/2020.2
  6) rhel7/global                        14) intel/bundles/complib/2020.2
  7) intel/compilers/2020.2              15) cmake/latest
  8) intel/mkl/2020.2                    16) rhel7/default-ccl
```

# Available modules

- You can list available modules as `$ module avail openmpi-4`

```
/usr/local/software/spack/current/share/spack/tcl/linux-rhel7-x86_64
openmpi-4.0.0-intel-17.0.8-tnwn5io  openmpi-4.0.5-gcc-11.1.0-rkwciv5
openmpi-4.0.1-gcc-5.4.0-p6kt5jp     openmpi-4.0.5-gcc-5.4-th76hhe
openmpi-4.0.1-gcc-7.2.0-myumaue     openmpi-4.0.5-gcc-8.4.1-2lpgttf
openmpi-4.0.2-gcc-5.4-mwnjuyg       openmpi-4.0.5-gcc-8.4.1-3xxjnxe
openmpi-4.0.2-gcc-5-gaullmv         openmpi-4.0.5-gcc-8.4.1-l7ihwk3
```

- These are compiled against different compilers/versions, and are (probably) only compatible with that specific one.
- The performance/feature-list/bug-status of each different version may vary.

# Loading modules

- In order to set up a specific compiler/MPI/etc. environment, use: `module load openmpi-4.0.5-gcc-8.4.1-l7ihwk3`
- as well as any other modules you need.
- If combined with `module purge` to reset your environment, this makes sure you have a reproducible set of compilers/libraries/tools loaded.
- These will also need to be loaded when you submit a job to the cluster queue.

# SLURM scripts

- CSD3 uses the SLURM queueing system; others do exist.
- You have to create a job script containing:

```
#! Job name
#SBATCH -J MySimulation_Name

#! Which project should be charged:
#SBATCH -A MYPROJECT-CPU

#! Which partition/cluster am I using?
#SBATCH -p cclake
#! How many nodes should be allocated? If not specified SLURM
#! assumes 1 node.
#SBATCH --nodes=1

#! How many tasks will there be in total? By default SLURM
#! will assume 1 task per node and 1 CPU per task.
#SBATCH --ntasks=56

#! How much wallclock time will be required?
#SBATCH --time=02:00:00
```

# SLURM script ctd

- Your SLURM script also has to contain the commands to load the correct environment (`module load`) and run your program.
- The template SLURM script contains suitable `mpirun` etc. commands to ensure your program launches correctly.
- On CSD3, see `/usr/local/Cluster-Docs/SLURM/slurm_submit.peta4-icelake`
- Once complete, submit your SLURM script to the queueing system: `sbatch ./submit_Euler_3D_Test` and wait...
- You can either examine the queue with `squeue` or wait for automated emails to come through.
- **Do not** repeatedly run `squeue` every 10s. The queue is rescheduled every 10-15 minutes, so checking repeatedly is pointless and overloads SLURM.

# Filesystems and Quotas

```
$ quota
Filesys     GB  quota  limit    grace   files  User/Grp
/home     18.3 50.0    55.0                    U:pmb39
/rds-d3  936.7  0.0    0.0     771124       0 G:pmb39
/rds-d7  580.8  1099.5 1209.5  771179 1048576 P:608
```

- On CSD3, the `/home` directories are backed-up.
- The RDS directories are not.
- The `/home` directories use an NFS filesystem and is suited to code, post-processed results, etc.
- The RDS directories use the Lustre filesystem which is scalable for parallel file-access.
- More details at `https://docs.hpc.cam.ac.uk/hpc/user-guide/io_management.html`